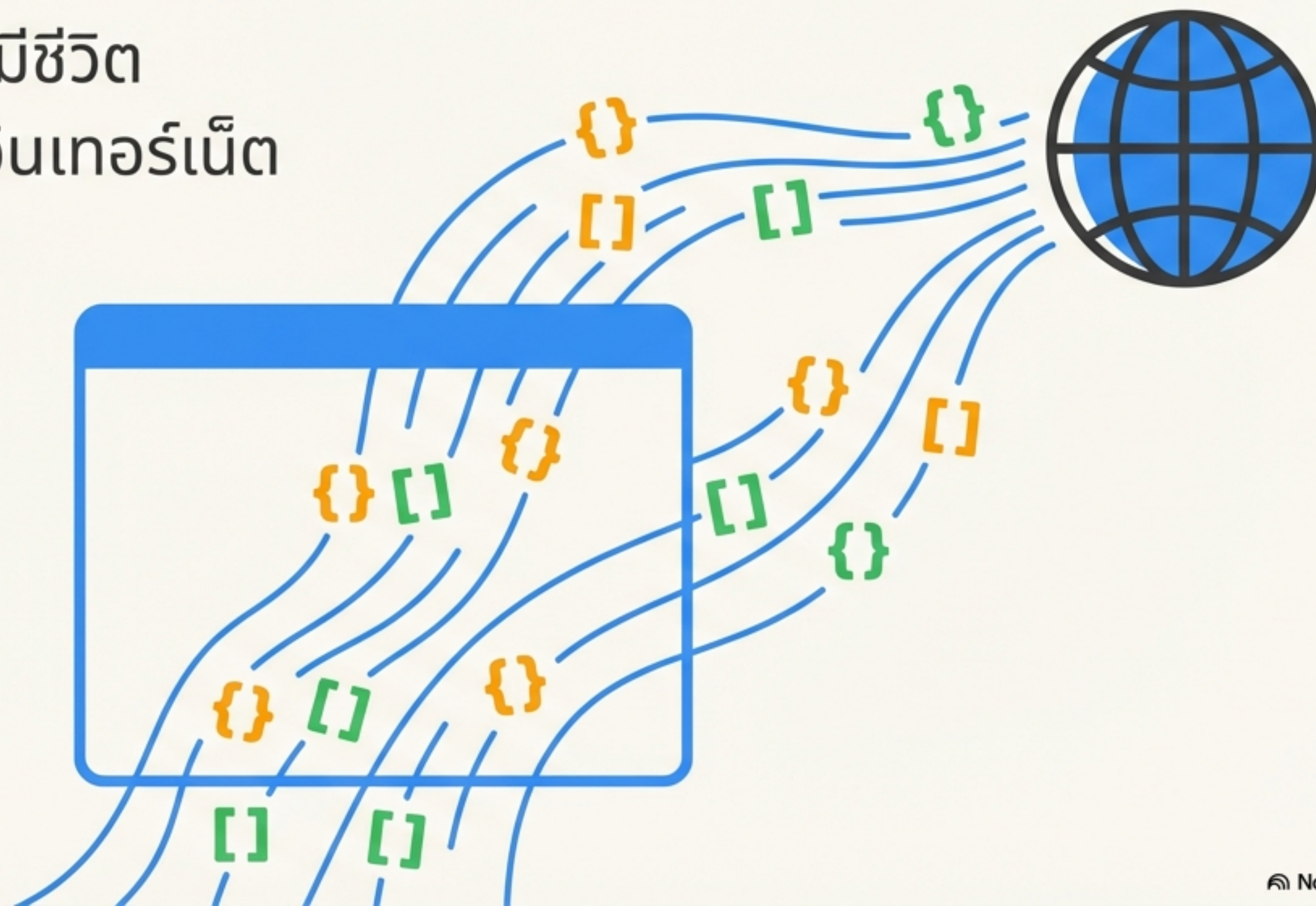
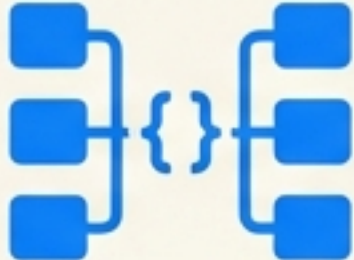


ชั่วโมงที่ 3: JSON และ Web API

เปลี่ยนหน้าเว็บธรรมดาให้มีชีวิต
ด้วยการดึงข้อมูลสดจากอินเทอร์เน็ต



เส้นทางการเรียนรู้ (Learning Journey)

- 

รู้จัก JSON: 'ภาษากลาง' ของข้อมูลบนเว็บคืออะไร?
รากบัวหล่อมความเกินไ้ต้องสมุดเล็กเมื่อนเอาารถดับ
- 

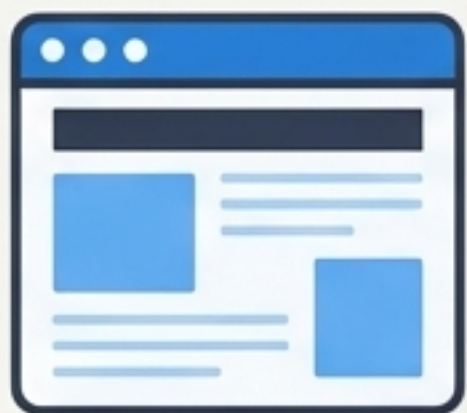
จัดการข้อมูลด้วย JavaScript: วิธีสร้างและใช้งานข้อมูล
ในโค้ดของเรา
- 

เชื่อมต่อโลกอินเทอร์เน็ตด้วย Web API: ช่องทาง 'สั่ง'
ข้อมูลจากเซิร์ฟเวอร์
- 

ลงมือสร้าง: สร้างเว็บแสดงข้อมูลจริงจาก API

JSON คืออะไร?

ภาษากลางในการ 'สนทนา' ระหว่างโปรแกรม



Client

{ "..." : "..." }

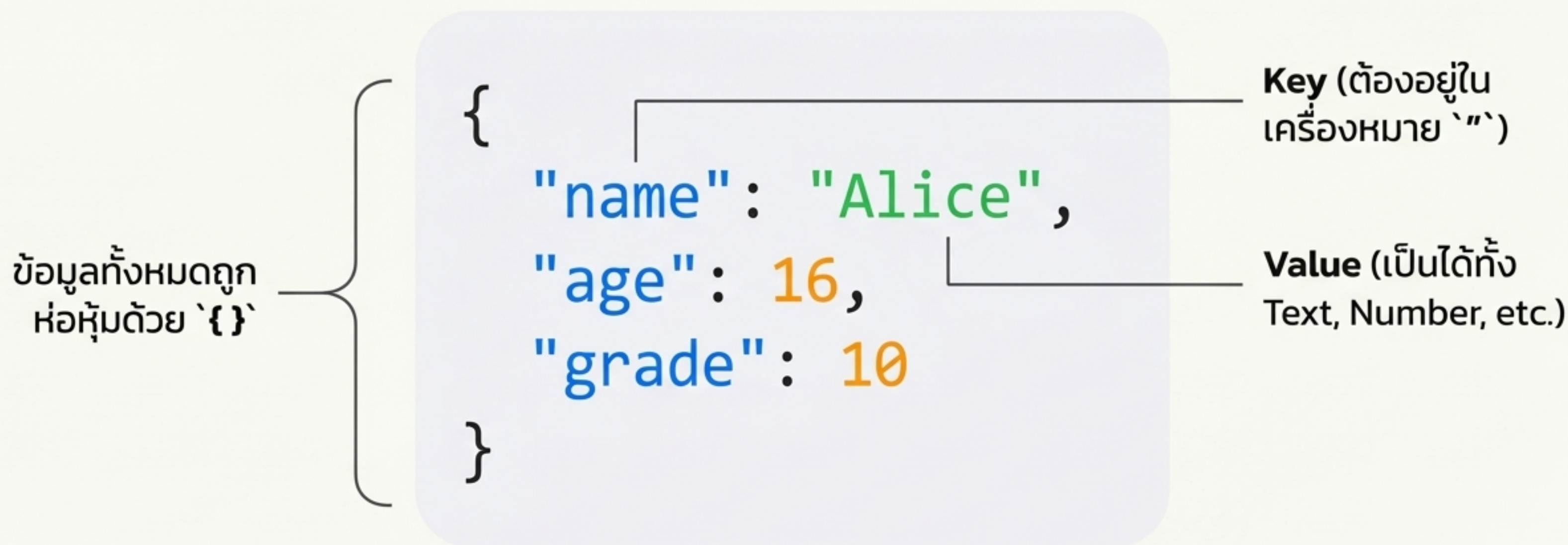


Server

- JSON (JavaScript Object Notation) คือรูปแบบข้อมูลมาตรฐานที่ใช้แลกเปลี่ยนข้อมูลระหว่าง Client (เช่น Browser) และ Server
- มีโครงสร้างที่มนุษย์อ่านเข้าใจง่าย และคอมพิวเตอร์ก็ประมวลผลง่ายเช่นกัน
- เป็นรูปแบบที่นิยมใช้กันอย่างแพร่หลายใน Web API ทั่วโลก

โครงสร้างพื้นฐานของ JSON

ประกอบด้วย `Key` (ชื่อข้อมูล) และ `Value` (ค่าของข้อมูล)



เทียบกับสิ่งที่คุ้นเคย: Python

ถ้าคุณรู้จัก Dictionary และ List ใน Python คุณก็เกือบจะรู้จัก JSON แล้ว!

แนวคิด	Python 	JSON 
Dictionary (เก็บข้อมูลแบบ Key-Value)	<pre>{"name": "Alice"}</pre>	<pre>{ "name": "Alice" }</pre>
List (เก็บข้อมูลเป็นลำดับ)	<pre>[1, 2, 3]</pre>	<pre>[1, 2, 3]</pre>

****Key Takeaway****: โครงสร้างหลักแทบจะเหมือนกัน! ทำให้ง่ายต่อการเรียนรู้

Object ใน JavaScript

โครงสร้างสำหรับเก็บข้อมูลที่เกี่ยวข้องกันภายใน JavaScript

หน้าตาคล้าย JSON มาก แต่เป็นสิ่งที่ JavaScript ‘เข้าใจ’ และทำงาน
ด้วยได้โดยตรง ไม่ใช่แค่ข้อความ (Text)

```
let student = {  
  name: "Bob",  
  age: 15,  
  grade: 10  
};
```


การเข้าถึงข้อมูลใน Object

ใช้เครื่องหมาย ``.` ตามด้วยชื่อ ``key`` เพื่อ 'ล้วง' ข้อมูลข้างใน

```
// จาก Object ที่เราสร้างไว้...  
console.log(student.name);  
console.log(student.age);
```

Developer Console

// ผลลัพธ์ที่แสดงใน Console:
> "Bob"

// ผลลัพธ์ที่แสดงใน Console:
> 15

ข้อมูลหลายชิ้น: Array ของ Object

จะอย่างไรถ้าเรามีนักเรียนหลายคน? → ใช้ Array `[ ]` ครอบ Object หลายๆ ตัวไว้

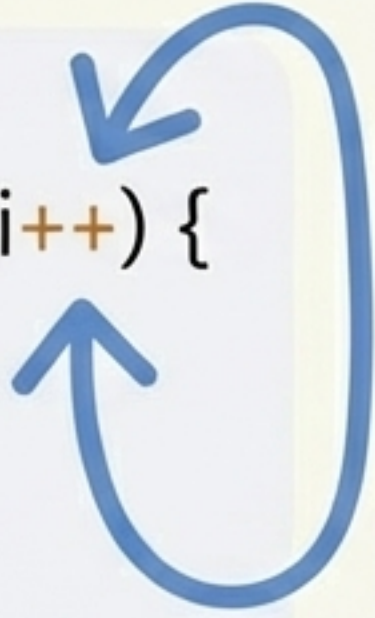
```
let students = [  
  { name: "Alice", score: 80 },  
  { name: "Bob", score: 65 },  
  { name: "Charlie", score: 45 }  
];
```

← Array ที่เก็บ Object
ของนักเรียนแต่ละคน

การวนลูปเพื่อแสดงข้อมูล (Looping)

ใช้ `for` loop เพื่อจัดการข้อมูลนักเรียนทีละคน

```
for (let i = 0; i < students.length; i++) {  
  // เข้าถึงชื่อของนักเรียนคนที่ i  
  console.log(students[i].name);  
}
```

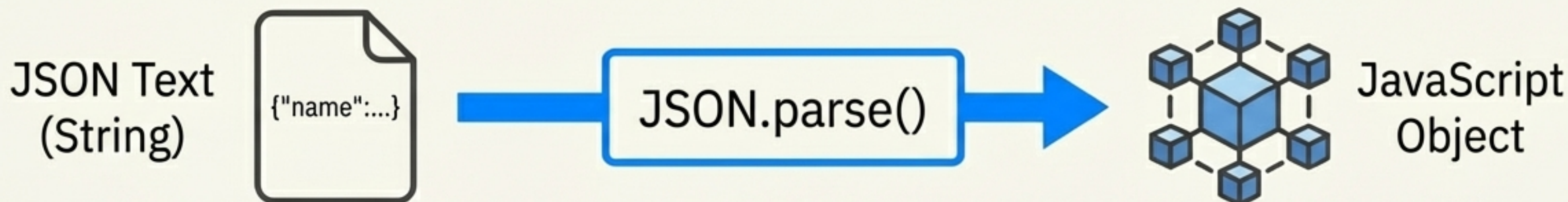


Developer Console

```
// ผลลัพธ์ที่แสดงใน Console:  
> "Alice"  
> "Bob"  
> "Charlie"
```


แปลงร่าง (1): JSON → JavaScript Object

เปลี่ยน 'ข้อความ' (Text) ที่ได้รับมา ให้กลายเป็น Object ที่ใช้งานได้



```
// สมมติว่านี่คือข้อมูล Text ที่เราได้รับจาก Server
let jsonText = '{"name":"Alice","age":16}';
// ใช้ JSON.parse() เพื่อแปลง Text ให้เป็น Object
let studentObject = JSON.parse(jsonText);
// ตอนนี้เราสามารถเข้าถึงข้อมูลข้างในได้แล้ว!
console.log(studentObject.name); // ผลลัพธ์: Alice
```


แปลงร่าง (2): JavaScript Object → JSON

เปลี่ยน Object ในโปรแกรมของเรา ให้กลับไปเป็น 'ข้อความ' (Text) เพื่อเตรียมส่งต่อ



JavaScript Object

JSON.stringify()



JSON Text (String)

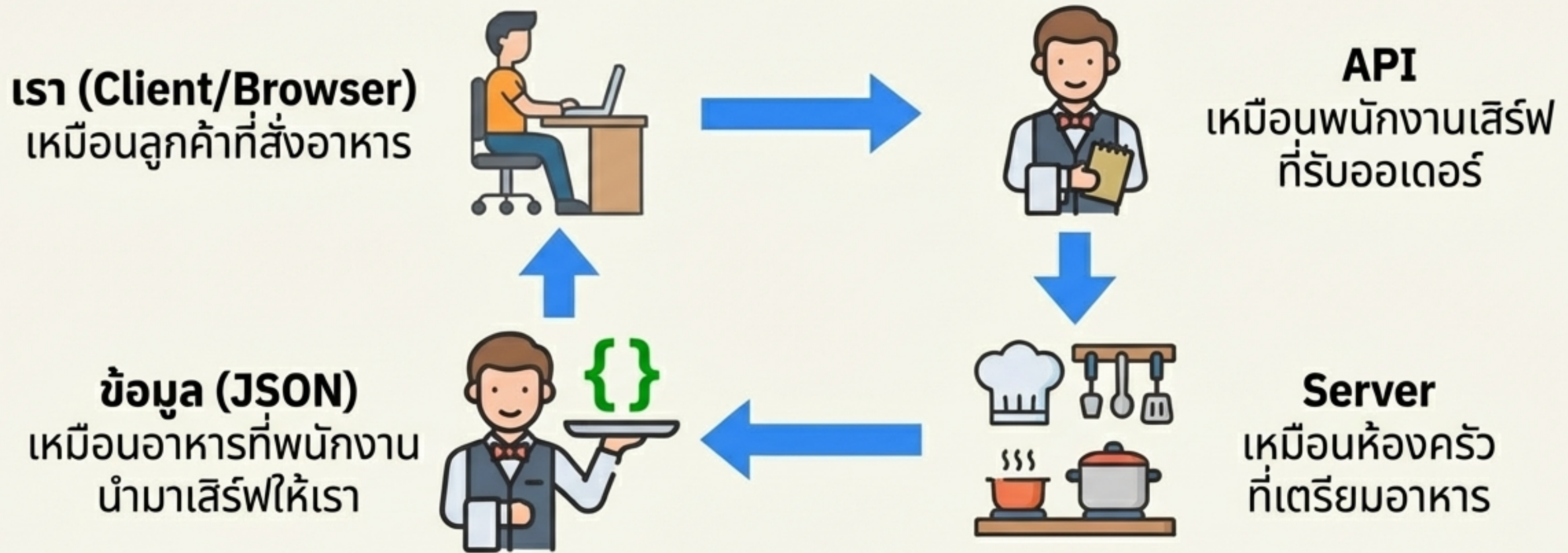
```
// Object ที่เรามีใน JavaScript
let student = { name: "Bob", age: 15 };

// ใช้ JSON.stringify() เพื่อแปลง Object เป็น JSON Text
let jsonString = JSON.stringify(student);

// ผลลัพธ์ที่ได้คือ String ที่พร้อมส่งไปให้ Server
console.log(jsonString); // ผลลัพธ์: '{"name":"Bob","age":15}'
```


Web API คืออะไร?

ช่องทางสำหรับ 'สั่ง' ข้อมูลจากเซิร์ฟเวอร์



Web API คือช่องทางให้โปรแกรมของเราสามารถเรียกใช้ **ข้อมูลหรือบริการ** จาก Server อื่นๆ ผ่านอินเทอร์เน็ตได้

วิธี 'สั่ง' ข้อมูล: `fetch()`

คำสั่งมาตรฐานใน JavaScript สำหรับเรียกใช้ Web API

fetch()

- `fetch()` เป็นฟังก์ชันสำหรับส่ง 'คำขอ' (Request) ไปยัง URL ของ API
- มันจะทำงานเบื้องหลัง (Asynchronous) เพื่อไม่ให้หน้าเว็บค้างระหว่างรอข้อมูล
- เมื่อ Server 'ตอบกลับ' (Response) เราจะนำข้อมูลที่ได้ (ส่วนใหญ่เป็น JSON) มาใช้งานต่อ

โครงสร้างพื้นฐานของ `fetch()`

- ```
fetch('https://api.example.com/data')
 .then(response => response.json())
 .then(data => {
 // ...นำ data (ข้อมูลที่แปลงแล้ว) ไปใช้งานที่นี่
 console.log(data);
 });
```
1. **`fetch(url)`**: ส่งคำขอไปที่ URL ของ API
  2. **`.then(response => response.json())`**:  
เมื่อ Server ตอบกลับ, ให้แปลงคำตอบนั้น  
เป็นข้อมูล JSON
  3. **`.then(data => { ... })`**:  
เมื่อแปลงข้อมูลเสร็จแล้ว, นำข้อมูล  
(ที่ตอนนี้เป็น JavaScript Object/Array)  
ไปใช้งานในส่วนนี้



# ลงมือทำ! (Hands-on Workshop)

**เป้าหมาย:** สร้างหน้าเว็บที่สามารถดึงและแสดงข้อมูลจากแหล่งข้อมูลจริง

## กิจกรรมที่ 1



สร้างหน้าเว็บที่อ่านข้อมูล JSON  
จากตัวแปรในโค้ด แล้วแสดงผล

## กิจกรรมที่ 2



สร้างหน้าเว็บที่มีปุ่มกด เพื่อเรียก  
Web API จริงๆ จากอินเทอร์เน็ต  
แล้วนำข้อมูลมาแสดงผล



# กิจกรรมที่ 1: อ่าน JSON จากตัวแปร

สร้างไฟล์ชื่อ `index.html` แล้วคัดลอกโค้ดทั้งหมดนี้ไปวาง จากนั้นเปิดไฟล์ด้วย Web Browser เพื่อดูผลลัพธ์

```
<!DOCTYPE html>
<html>
<head>
 <title>JSON Example</title>
</head>
<body>
 <h1>Student Data</h1>
 <div id="output"></div>

 <script>
 const jsonText = '[{"name":"Alice","score":80}, {"name":"Bob","score":65}, {"name":"Charlie","score":45}]';
 const students = JSON.parse(jsonText);

 let htmlContent = '';
 for (let i = 0; i < students.length; i++) {
 htmlContent += '${students[i].name}: ${students[i].score} คะแนน';
 }
 htmlContent += '';

 document.getElementById('output').innerHTML = htmlContent;
 </script>
</body>
</html>
```



## กิจกรรมที่ 2: เรียก Web API จริง

สร้างไฟล์ `index.html` ใหม่ แล้วคัดลอกโค้ดทั้งหมดนี้ไปวาง เปิดไฟล์ด้วย Browser แล้วกดปุ่ม 'Load Users'  
API Endpoint: `https://jsonplaceholder.typicode.com/users`

```
<!DOCTYPE html>
<html>
 <head>
 <title>Web API Example</title>
 </head>
 <body>
 <h1>User List from API</h1>
 <div id="users"></div>
 <button onclick="loadUsers()">Load Users</button>

 <script>
 function loadUsers() {
 fetch('https://jsonplaceholder.typicode.com/users')
 .then(response => response.json())
 .then(data => {
 let htmlContent = '';
 for (let i = 0; i < data.length; i++) {
 htmlContent += '${data[i].name} (${data[i].email})';
 }
 htmlContent += '';
 document.getElementById('users').innerHTML = htmlContent;
 });
 }
 </script>
 </body>
</html>
```



# ผลลัพธ์ที่คาดหวัง

กิจกรรมที่ 1

กิจกรรมที่ 2 (หลังจากกดปุ่ม)

## Student Data

- Alice: 80 คะแนน
- Bob: 65 คะแนน
- Charlie: 45 คะแนน

## User List from API

- Leanne Graham (Sincere@april.biz)
- Ervin Howell (Shanna@melissa.tv)
- Clementine Bauch (Nathan@yesenia.net)
- ... (และข้อมูลผู้ใช้อื่นๆ ต่อไป)

Load Users



# ลองไปต่อด้วยตัวเอง (Challenge)

คุณสามารถทำอะไรกับโค้ดในกิจกรรมที่ 2 ได้อีกบ้าง?



- **เปลี่ยน Endpoint:** ลองเปลี่ยน URL ใน `fetch` ไปเป็น `.../todos` หรือ `.../posts` แล้วดูว่าข้อมูลที่ได้ต่างกันอย่างไร



- **เลือกแสดงข้อมูล:** แก้ไขโค้ดให้แสดงเฉพาะ 'ชื่อ' หรือ 'อีเมล' อย่างเดียว



- **เปลี่ยนการแสดงผล:** ลองเปลี่ยนจากการแสดงผลใน `<ul><li>` (list) ไปเป็นการแสดงผลในตาราง `<table>`



- **กรองข้อมูล:** เพิ่มเงื่อนไข `if` ใน `for` loop เพื่อแสดงเฉพาะข้อมูลที่ต้องการ (เช่น แสดงเฉพาะผู้ใช้ที่ชื่อขึ้นต้นด้วย 'E')



# สรุปสิ่งที่คุณทำได้แล้ววันนี้

You are now a Web Data Wrangler!



- ✓ อธิบายแนวคิดของ JSON และ Web API ได้
- ✓ สร้างและใช้งาน Object และ Array ใน JavaScript
- ✓ แปลงข้อมูลไปมาระหว่าง JSON Text และ JavaScript Object
- ✓ เรียกข้อมูลจาก Web API จริงๆ ด้วยคำสั่ง `fetch()`
- ✓ นำข้อมูลที่ได้รับมาแสดงผลบนหน้าเว็บได้สำเร็จ